

# De Bruijn Notation and Recursion in the Faust Compiler

## De Bruijn Notation and Recursion in the Faust Compiler

### Abstract

This note synthesizes two distinct ideas that are often conflated: on the one hand, de Bruijn notation as the standard nameless representation of bound variables; on the other hand, its use in Faust as an intermediate representation for feedback groups produced by the `~` operator. The central point is this: Faust does not use de Bruijn indices as a merely theoretical device, but as a propagation IR tailored to multi-output recursive groups, causalized by `delay1`, before conversion to a more convenient symbolic form for later passes.

The claim defended here is deliberately nuanced. The use of de Bruijn outside pure lambda calculus is not, by itself, a Faust invention: the literature describes de Bruijn-based representations for multiple binders, recursively scoped syntaxes, and other non-trivial binding disciplines. What is much more specific to Faust is the precise adaptation to signal-feedback groups, with slot projections and a staged pipeline of `boxes`  $\rightarrow$  `de Bruijn signals`  $\rightarrow$  `symbolic signals`  $\rightarrow$  `FIR`.

### 1. De Bruijn notation in its classical use

In his 1972 foundational paper, Nicolaas Govert de Bruijn proposed replacing bound variable names with integers encoding the distance to the corresponding binder. The stated goal was clear: avoid the complications of substitution and alpha-conversion in a representation meant for mechanized manipulation.

In its classical form:

```
lambda x. x           -> lambda. 1
lambda x. lambda y. x -> lambda. lambda. 2
lambda x. lambda y. y -> lambda. lambda. 1
```

The semantic principle is straightforward:

- 1 denotes the nearest binder;
- 2 denotes the next binder outward;
- references are therefore expressed by depth rather than by name.

This representation has two well-known consequences:

1. alpha-equivalence becomes structural equality;
2. substitution and lifting become purely structural operations.

In more recent literature, de Bruijn representation is no longer restricted to unary lambda calculus. Keuchel and Jeuring explicitly show that well-scoped de Bruijn representations can also describe multiple binders, sequential scopes, and recursive scopes. I therefore infer that the general idea of "de Bruijn beyond pure lambda calculus" is not Faust-specific; what is Faust-specific is the kind of structure that Faust chooses to encode this way.

## 2. What Faust does with de Bruijn notation

In Faust, source programs do not contain `DEBRUIJNREC` or `DEBRUIJNREF` nodes. Recursion is written at the box-language level, via the `~` operator or, in the box API, via `boxRec`. The official Faust documentation highlights two important facts:

- the semantic phase translates the program into signals by symbolic propagation;
- recursion automatically inserts a one-sample delay to guarantee causality.

The internal IR then uses two main forms:

| Form                            | Role                                       |
|---------------------------------|--------------------------------------------|
| <code>DEBRUIJNREC(body)</code>  | binder for one recursive group             |
| <code>DEBRUIJNREF(level)</code> | reference to one enclosing recursive group |

Faust's adaptation differs from standard lambda calculus in one decisive way: the binder does not bind a single variable, but a group of outputs. A recursive reference is therefore not useful on its own; it is almost always combined with a slot projection:

```
proj(i, DEBRUIJNREF(1))
```

The central causal-feedback pattern is then:

```
delay1(proj(i, DEBRUIJNREF(1)))
```

In other words, Faust does not use de Bruijn indices to name lambda-term variables, but to identify the current recursive group inside a multi-output signal graph. In my view, that is the most interesting specificity of this IR.

## 3. From recursive boxes to de Bruijn form in propagate

### 3.1 General scheme

At the box level, `A ~ B` constructs a recursive composition. If

- `A : Li -> Lo`
- `B : Ri -> Ro`

then the composition is well-formed when `Ri <= Lo` and `Ro <= Li`.

In the Rust port, the `FlatNodeKind::Rec(left, right)` branch in `crates/propagate/src/lib.rs` follows essentially this scheme:

```
l0 = [ delay1(proj(i, DEBRUIJNREF(1))) ]   for i = 0..Ri-1
l1 = propagate(right, l0)
l2 = propagate(left, l1 ++ lift(inputs))
group = DEBRUIJNREC(list(l2))
```

```
output[i] =
  if aperture(l2[i]) > 0 then proj(i, group)
  else l2[i]
```

Three ingredients matter structurally:

- the feedback placeholders `delay1(proj(i, DEBRUIJNREF(1)))`;
- `lift` on free references when entering a nested recursion;
- the `aperture` test, which decides whether a branch is genuinely recursive.

### 3.2 Simple example

Minimal example:

```
process = + ~ *(0.5);
```

The compiler first builds a feedback seed:

```
delay1(proj(0, DEBRUIJNREF(1)))
```

Then the feedback path  $\ast(0.5)$  transforms it, and the main body  $+$  builds, schematically:

```
body0 =  
  add(  
    delay1(proj(0, DEBRUIJNREF(1))) * 0.5,  
    input(0)  
  )
```

```
group = DEBRUIJNREC([body0])  
out0 = proj(0, group)
```

The important point is that `DEBRUIJNREF(1)` does not mean "the variable  $W$  of the current group". It means "the nearest recursive group"; slot selection is deferred to `proj(0, ...)`.

### 3.3 Nested recursive forms

A representative shape is:

```
inner = + ~ *(0.5);  
process = inner ~ *(0.25);
```

Here, the recursion defined by `inner` is itself placed under a new recursive binder. The consequence is standard de Bruijn behavior: every free reference coming from the outer scope must be lifted by one level so that it is not captured by the inner binder.

Schematically, one obtains a structure of the form:

```
DEBRUIJNREC(                                <- outer group  
  ...  
  DEBRUIJNREC(                              <- inner group  
    pair(  
      DEBRUIJNREF(1),                      <- points to the inner group  
      DEBRUIJNREF(2)                      <- points to the outer group  
    )  
  )  
  ...  
)
```

The Rust port encodes this through `liftn(...)`:

```
liftn(DEBRUIJNREF(n), threshold) =  
  DEBRUIJNREF(n)    if n < threshold  
  DEBRUIJNREF(n+1) otherwise
```

This very short rule actually hides three separate ideas:

- **threshold** should be read as "from which level onward the reference is still free relative to the scope we are about to enter";
- a reference whose level is strictly smaller than **threshold** is already bound inside the current subtree, so it must not be changed;

- a reference whose level is greater than or equal to `threshold` remains free with respect to the new scope, so it must be shifted by one level to keep pointing to the same logical binder after insertion of the new `DEBRUIJNREC`.

In other words, `lift` does not "make everything deeper". It lifts only the free part of the subtree.

Seen as a complete structural algorithm, `liftn` behaves more like this:

```
liftn(node, threshold):
  if node = DEBRUIJNREF(level):
    return DEBRUIJNREF(level)    if level < threshold
    return DEBRUIJNREF(level + 1) otherwise

  if node = DEBRUIJNREC(body):
    return DEBRUIJNREC(liftn(body, threshold + 1))

  otherwise:
    rebuild node by applying liftn to each child
```

The `threshold + 1` is the key point. Descending under an inner recursive binder means that one more level becomes locally bound. The criterion for "freeness" must therefore shift as well.

### 3.3.1 Minimal example: free reference versus already-bound reference

Consider the following subtree just before entering a new recursion:

```
pair(
  DEBRUIJNREF(1),
  DEBRUIJNREC(DEBRUIJNREF(1))
)
```

Applying `liftn(..., 1)` gives:

```
pair(
  DEBRUIJNREF(2),
  DEBRUIJNREC(DEBRUIJNREF(1))
)
```

The difference between the two branches is essential:

- in the left branch, `DEBRUIJNREF(1)` is free with respect to the new scope, so it must become `DEBRUIJNREF(2)`;
- in the right branch, `DEBRUIJNREF(1)` is already captured by the local `DEBRUIJNREC`, so it must remain 1.

This example shows why `lift` must not be understood as "add 1 to all references". That would be wrong: it would break references that are already bound inside inner subgroups.

### 3.3.2 Intuitive Faust-pipeline example

Suppose a value coming from an outer recursion already contains:

```
delay1(proj(0, DEBRUIJNREF(1)))
```

and this value is reused inside a new `Rec`. Without lifting, `DEBRUIJNREF(1)` would now be re-read as "the new inner group", even though it originally meant "the already existing outer group".

After `liftn(..., 1)`, we obtain:

```
delay1(proj(0, DEBRUIJNREF(2)))
```

and lexical meaning is preserved:

- `DEBRUIJNREF(1)` now denotes the newly created inner group;
- `DEBRUIJNREF(2)` continues to denote the old outer group.

When a subtree coming from an outer recursion enters an inner recursion, a free `DEBRUIJNREF(1)` therefore becomes `DEBRUIJNREF(2)`. Without that shift, the reference would be incorrectly captured by the new `DEBRUIJNREF`.

### 3.3.3 Why Faust must lift in two places

In `propagate`, this operation is applied to two families of objects:

- the `inputs` injected into the `left` body of the `Rec`;
- the values stored in `slot_env`.

The second point is easy to miss, but it is essential. A value produced by box abstraction, a local definition, or a closure may itself contain `DEBRUIJNREF` nodes coming from an outer loop. If that value is reinjected unchanged into an inner loop, it will be silently captured by the wrong binder.

So the deeper role of `lift` in Faust is not merely "to make nested loops work". More precisely, it maintains the following invariant:

introducing a new recursive group must never accidentally change the logical binder targeted by a pre-existing free reference.

### 3.4 Multi-output recursion and mutual recursion

In Faust, mutual recursion is a special case of multi-output recursion, not a synonym for it. A regression example already present in the corpus is:

```
import("stdfaust.lib");

feedback = si.bus(2) ~ ((*0.5), *(0.25));
process = feedback;

DEBRUIJNREC([
  body0 = delay1(proj(0, DEBRUIJNREF(1))) * 0.5,
  body1 = delay1(proj(1, DEBRUIJNREF(1))) * 0.25
])
```

Strictly speaking, this example is mainly a multi-output case: each channel mostly feeds back into itself. But from the compiler's point of view, the truly mutually recursive case introduces no new mechanism; it only changes which projections appear in each body. For example:

```
import("stdfaust.lib");

process = si.bus(2) ~ ((*0.5), *(0.25)) : ro.cross(2));
which lowers schematically to:

DEBRUIJNREC([
  body0 = delay1(proj(1, DEBRUIJNREF(1))) * 0.25,
  body1 = delay1(proj(0, DEBRUIJNREF(1))) * 0.5
])
```

This time the two outputs depend on each other, not only on themselves.

The key point is therefore this: in Faust, "mutual recursion" is not a second device next to ordinary recursion; it is the same representation, but with a list-shaped group body and slot projections.

## 4. Degenerate recursive forms: production, diagnosis, simplification

A recursive form is called degenerate when the material recursive group still exists, but one or more of its outputs no longer actually depend on feedback. This happens when some branches of the recursive path:

- explicitly discard their recursive input;
- or become closed after propagation and simplification.

The corpus contains a representative case:

```
import("stdfaust.lib");

N = 8;
gain = hslider("gain", 0.5, 0.0, 0.99, 0.01);
process = si.bus(N) ~ (!, !, !, !, !, !, !, *(gain));
```

Here, channels 0..6 discard their recursive input via `!`; only channel 7 remains genuinely recursive. The conceptual tool used to detect this is `aperture`, defined as the maximum free de Bruijn reference level in a subtree:

```
aperture(DEBRUIJNREF(k))      = k
aperture(DEBRUIJNREC(body))   = aperture(body) - 1
aperture(other node)         = max(aperture(children))
```

Interpretation:

- `aperture > 0`: the branch is still open over the recursive group;
- `aperture <= 0`: the branch is closed at that boundary.

In `propagate`, a closed output is already no longer re-emitted as `proj(i, group)`. But this does not necessarily remove all structural degeneracy from the group. In the classic C++ pipeline, a more aggressive pass, `inlineDegenerateRecursions()`, can compact a group and keep only the genuinely recursive bodies. This creates a subtle problem: the logical projection index may remain the original one, while the physical arity of the group has shrunk.

Typical example:

```
before compaction: proj(7, SYMREC([b0, ..., b7]))
after compaction:  proj(7, SYMREC([b7]))
```

The group now has only one physical body, but the projection is still 7. For a backend that models recursive slots as a `Vec`, that is an unstable or even invalid form.

In the current Rust fast lane, the adopted simplification is narrower: in `signal_prepare`, every projection targeting a unary symbolic group is canonicalized to `proj(0, group)`. The point is therefore not merely cosmetic. Simplifying degenerate forms is useful to:

- keep slot indices dense;
- stabilize typing and FIR lowering;
- avoid "projection index out of bounds" failures.

## 5. Converting de Bruijn form to symbolic form

### 5.1 Why convert

De Bruijn form is excellent during propagation:

- no name generation is needed;
- scopes are correct by construction;
- structural sharing in the arena is maximized.

However, it is not especially readable and is not very convenient for later passes. Mentally counting binder depths inside a shared signal DAG is tolerable for `propagate`, much less so for typing, FIR preparation, recursive-group lowering, and diagnostics.

Symbolic form therefore replaces:

| De Bruijn form     | Symbolic form     |
|--------------------|-------------------|
| DEBRUIJNREC(body)  | SYMREC(var, body) |
| DEBRUIJNREF(level) | SYMREF(var)       |

## 5.2 Algorithm

The `de_bruijn_to_sym(...)` algorithm, shared between the historical C++ compiler and the Rust port, is conceptually:

```
convert(node):
  if node = DEBRUIJNREC(body):
    var = fresh("W")
    body1 = substitute(body, level=1, replacement=SYMREF(var))
    body2 = convert(body1)
    return SYMREC(var, body2)

  if node = DEBRUIJNREF(level):
    error: the converted root was open

  otherwise:
    recursively rebuild the node
```

The subtle point is substitution:

```
substitute(node, level, repl):
  if aperture(node) < level:
    return node
  if node = DEBRUIJNREF(level):
    return repl
  if node = DEBRUIJNREC(body):
    return DEBRUIJNREC(substitute(body, level + 1, repl))
  otherwise:
    rebuild
```

When descending under a nested `DEBRUIJNREC`, the searched level becomes `level + 1`, exactly as in `liftn`. That is what makes correct conversion of nested trees possible.

## 5.3 Nested example

The test `crates/tlib/tests/recursive_trees.rs` encodes the minimal case:

```
DEBRUIJNREC(
  DEBRUIJNREC(
    pair(DEBRUIJNREF(1), DEBRUIJNREF(2))
  )
)
```

The conversion yields:

```
SYMREC(W0,
  SYMREC(W1,
```

```

    pair(SYMREF(W1), SYMREF(W0))
  )
)
```

The expected lexical logic is recovered:

- DEBRUIJNREF(1) in the inner group becomes SYMREF(W1);
- DEBRUIJNREF(2) in that same group becomes SYMREF(W0).

## 5.4 Usefulness for later compiler phases

In the current Rust port, `prepare_signals_for_fir(...)` clones the whole output forest, applies `de_bruijn_to_sym(...)` to the complete list, then performs unary-group normalization, reduced typing, and finally FIR preparation.

The FIR lowerer then expects groups of the form:

```

SYMREC(var, body_list)
SYMREF(var)
```

and no longer raw DEBRUIJNREC / DEBRUIJNREF nodes. This choice has three practical benefits:

- it makes the identity of the recursive group explicit;
- it lets `signal_fir` decode the body list of a symbolic group directly;
- it establishes a clean pipeline boundary: after preparation, the backend no longer needs to reason in terms of lexical depths.

In short, the conversion is not merely cosmetic. It is a representation change that separates scope logic, which is useful during propagation, from backend consumption logic, which is useful during lowering.

## 6. Conclusion

In Faust, de Bruijn notation plays a more concrete role than in many purely theoretical presentations: it serves as a working form for building feedback groups that are correct by construction, even in the presence of nesting, multi-output groups, and structural sharing.

The most important point is not simply "Faust uses de Bruijn", which would be too general, but rather:

- Faust treats recursion as a group binder, not as a single variable;
- recursive references are addressed first by depth, then by slot projection;
- degenerate forms impose a canonicalization discipline;
- the final conversion to SYMREC / SYMREF isolates downstream passes from scope complexity.

From that perspective, de Bruijn form in Faust is both classical in principle and highly specialized in compiler use.

## References

### External sources

- N. G. de Bruijn, "Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church-Rosser theorem", 1972. <https://research.tue.nl/en/publications/lambda-calculus-notation-with-nameless-dummies-a-tool-for-automat-2/>
- Faust Documentation, "Using the box API", sections "Faust compiler structure" and "Defining recursive signals". <https://faustdoc.grame.fr/tutorials/box-api/>

### References on de Bruijn usage beyond pure lambda calculus

- Type theory / logical frameworks / Automath:

- Fairouz Kamareddine, Alejandro Rios, "Pure Type Systems with de Bruijn Indices", *The Computer Journal*, 45(2), 2002. <https://doi.org/10.1093/comjnl/45.2.187>
- J.H. Geuvers, R.P. Nederpelt, "N.G. de Bruijn's contribution to the formalization of mathematics", *Indagationes Mathematicae*, 24(4), 2013. <https://doi.org/10.1016/j.indag.2013.09.003>
- Multiple binders, sequential scopes, and recursive scopes:
  - Steven Keuchel, Johan T. Jeuring, "Generic Conversions of Abstract Syntax Representations", WGP 2012. <https://ics-archive.science.uu.nl/research/techreps/repo/CS-2012/2012-009.pdf>
- Higher-order rewriting and metaterms:
  - Eduardo Bonelli, Delia Kesner, Alejandro Rios, "A de bruijn notation for higher-order rewriting", RTA 2000. [https://doi.org/10.1007/10721975\\_5](https://doi.org/10.1007/10721975_5)
  - Eduardo Bonelli, Delia Kesner, Alejandro Rios, "de Bruijn Indices for Metaterms", *Journal of Logic and Computation*, 15(6), 2005. <https://doi.org/10.1093/logcom/exi051>
- First-order logic with quantifiers:
  - Manuel Eberl Wehr, Daniel Kirst, "Material dialogues for first-order logic in constructive type theory: extended version", *Mathematical Structures in Computer Science*, 2024. <https://www.cambridge.org/core/journals/mathematical-structures-in-computer-science/article/material-dialogues-for-firstorder-logic-in-constructive-type-theory-extended-version/17E117C76725C980F4EAA68F76203C77>
- Process calculi / mechanized metatheory:
  - Roly Perera, James Cheney, "Proof-relevant pi-calculus: a constructive account of concurrency and causality", *Mathematical Structures in Computer Science*, 2017. <https://www.cambridge.org/core/journals/mathematical-structures-in-computer-science/article/proofrelevant-calculus-a-constructive-account-of-concurrency-and-causality/952DC4F0B460B604B3F9047FC41FE04A>

## Internal repository sources

- recursion-debruijn-lowering-en.md
- flatnode-rec-to-signals-en.md
- crates/propagate/src/lib.rs
- crates/tlib/src/recursion.rs
- crates/transform/src/signal\_prepare.rs
- crates/transform/src/signal\_fir/module.rs
- tests/corpus/rep\_79\_multi\_output\_recursion.dsp
- tests/corpus/rep\_71\_degenerate\_unary\_recursion.dsp